One-Shot and Few-Shot Learning: Concepts, Mathematical Foundations, and Prediction Strategies

1 Introduction to One-Shot Learning

1.1 Definition

One-shot learning refers to the ability of a model to recognize or classify new categories using only **one (or very few) examples** per class. Unlike traditional deep learning approaches, which require large labeled datasets, one-shot learning enables generalization from limited data.

1.2 Motivation

- Humans can recognize new objects after seeing them **once** (e.g., identifying a new type of animal).
- Deep learning models often require thousands of labeled images per category.
- One-shot learning is useful in scenarios where data collection is expensive (e.g., medical imaging, rare species classification).

2 Approaches to One-Shot Learning

One-shot learning is often implemented using **metric learning** or **generative models**.

2.1 Metric Learning Approach

- The model does not learn explicit class labels but instead learns a **distance function**.
- Given two inputs, it predicts whether they belong to the **same or different class**.
- Common architectures: **Siamese Networks, Triplet Networks, and Prototypical Networks**.

2.2 Generative Model Approach

- Instead of learning to compare samples, the model **learns the distribution** of the data and generates new examples.
- Examples: **Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs), Diffusion Models**.

3 Prototypical Networks for One-Shot Learning

3.1 Concept

- Prototypical Networks classify new samples by comparing them to a set of **class prototypes**.
- A **prototype** is the mean embedding of all samples in a class.
- Instead of comparing individual pairs like in Siamese Networks, Prototypical Networks compute a **class representation** and classify based on distance.

3.2 Mathematical Formulation

For a class c, the **prototype** is defined as:

$$p_c = \frac{1}{|S_c|} \sum_{x \in S_c} f(x)$$

where:

- p_c is the **prototype** for class c.
- S_c is the set of support examples for class c.
- $f(x)$ is the feature embedding extracted from the neural network.

A new query sample x_q is assigned to the **closest prototype** using:

$$\hat{y} = \arg\min_c d(f(x_q), p_c)$$

where $d(x, y)$ is a distance metric (e.g., Euclidean distance).

3.3 Prediction with Prototypical Networks

1. Compute the **prototype vector** for each class.
2. Compute the **distance** between the query point and each prototype.
3. Assign the query to the **class with the closest prototype**.

3.4 Use Cases

- **Handwritten character recognition** (Omniglot dataset).
- **Few-shot image classification** (miniImageNet dataset).
- **Medical diagnosis** (classifying rare diseases with limited examples).

4 Siamese Networks for One-Shot Learning

4.1 Concept

- Uses **twin neural networks** with **shared weights**.
- Each network extracts feature embeddings, and a similarity function (e.g., Euclidean distance) is applied.
- The model is trained using **contrastive loss**.

4.2 Contrastive Loss Function

For a dataset of N pairs (x_i, x_j), the loss is defined as:

$$L = \sum_{i=1}^{N} \left(y_{ij} d(x_i, x_j)^2 + (1 - y_{ij}) \max(0, m - d(x_i, x_j))^2 \right)$$

where:

- x_i, x_j are two input samples.
- $y_{ij} = 1$ if they belong to the **same class**, otherwise $y_{ij} = 0$.
- $d(x_i, x_j) = \|f(x_i) - f(x_j)\|$ is the feature embedding distance.
- m is a **margin** to push dissimilar pairs apart.

5 Triplet Loss for Metric Learning

5.1 Concept

- Instead of using pairs, **Triplet Networks** learn from **triplets**: (Anchor, Positive, Negative).
- Goal: Ensure that the anchor is **closer to the positive** than the negative by a margin α.

5.2 Triplet Loss Function

For a dataset of N triplets (A_i, P_i, N_i), the loss is:

$$L = \sum_{i=1}^{N} \max(0, d(A_i, P_i) - d(A_i, N_i) + \alpha)$$

where:

- A_i is the **anchor** (reference sample).
- P_i is a **positive** (same class as A_i).
- N_i is a **negative** (different class from A_i).
- $d(x, y) = \|f(x) - f(y)\|$ is the distance function.
- α is a **margin** to enforce separation.

6 Comparing Siamese Networks, Triplet Networks, and Prototypical Networks

Approach	Goal	Loss Function	Data Requiremen
Siamese Networks	Compare two inputs	Contrastive Loss	Pairs of similar/dissimila
Triplet Networks	Learn ranking of similarities	Triplet Loss	Triplets (Anchor, Positive, N
Prototypical Networks	Compare with class prototypes	Distance-based loss	Multiple examples per c

7 Summary

- **One-shot learning** enables classification with very few examples.
- **Siamese Networks** learn similarity via **contrastive loss**.
- **Triplet Networks** use **triplet loss** to push different classes apart.
- **Prototypical Networks** classify based on **class prototypes**.
- Applications include **facial recognition, signature verification, and medical imaging**.

8 Few-Shot Learning

8.1 Why Do We Need Few-Shot Learning?

Traditional deep learning models require **thousands of labeled examples per class** to generalize well. However, in many real-world applications, collecting such data is **expensive or impractical**. Few-shot learning is useful when:

- **Medical imaging**: Diagnosing rare diseases with very limited patient data.
- **Facial recognition**: Recognizing a person with only a few images.
- **Speech and NLP tasks**: Understanding new words or commands with minimal data.
- **Robotics**: Enabling robots to adapt quickly to new tasks.

8.2 The Core Idea Behind Few-Shot Learning

Instead of training a model with **many examples per class**, few-shot learning **trains a model to quickly adapt** using only a **few** labeled examples.

- **N-way, K-shot learning**:
  - **N-way**: The number of classes in each learning episode.
  - **K-shot**: The number of labeled examples per class.
- Example: A **5-way, 1-shot** setting means the model sees **5 classes** and **only 1 example per class** before making a prediction.

8.3 Approaches to Few-Shot Learning

Few-shot learning is typically implemented using the following approaches:

8.3.1 Metric-Based Learning

- The model learns a **distance function** to compare query samples with known examples.
- Uses similarity measures like **Euclidean distance, cosine similarity**, or a learned function.

Common models:

1. **Siamese Networks**: Compare pairs of images and use contrastive loss.

2. **Triplet Networks**: Learn an embedding space by pushing similar samples closer and dissimilar samples apart.

3. **Prototypical Networks**: Compute a **class prototype** as the mean embedding of available examples and classify new samples based on distance.

8.3.1.1 Mathematical Formulation of Prototypical Networks

- The **prototype (centroid)** for each class c is computed as:

$$p_c = \frac{1}{|S_c|} \sum_{x \in S_c} f(x)$$

- A query sample x_q is classified based on its **distance to the prototypes**:

$$P(y = c|x_q) = \frac{\exp(-d(f(x_q), p_c))}{\sum_{c'} \exp(-d(f(x_q), p_{c'}))}$$

- The loss function is the **negative log-likelihood**:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \log P(y_i = c|x_i)$$

8.3.2 Optimization-Based Learning (Meta-Learning)

Instead of learning to classify directly, the model **learns how to learn**.

- **Idea**: Train a model that can adapt to new classes with very few examples.
- The model is trained on multiple small **tasks (episodes)** that simulate a few-shot scenario.

Common meta-learning algorithms:

1. **Model-Agnostic Meta-Learning (MAML)**: Finds an initial model that can quickly adapt to new tasks with minimal updates.
2. **Reptile**: A simpler alternative to MAML that optimizes for fast adaptation.
3. **Memory-Augmented Networks (MANNs)**: Use external memory to store learned representations and retrieve them for new tasks.

8.3.3 Data Augmentation & Generative Approaches

- When labeled data is extremely limited, models can **generate new synthetic examples**.
- Common techniques:
  - **GANs (Generative Adversarial Networks)**: Generate synthetic data for unseen classes.
  - **Data augmentation**: Apply transformations (rotation, cropping, flipping, noise injection) to artificially increase dataset size.
  - **Few-shot image synthesis**: Uses a **meta-learning framework** to generate high-quality synthetic samples.

Approach	Key Idea	Common Models
Metric-Based	Learn a distance function	Siamese Networks, Triplet Networks, Prototypical Netwo
Optimization-Based	Learn how to adapt quickly	MAML, Reptile, Meta-Learning
Data Augmentation	Generate synthetic samples	GANs, Augmentation techniques

Table 1: Comparison of Few-Shot Learning Approaches

8.4 Comparison of Few-Shot Learning Methods

8.5 Few-Shot Learning in Practice

8.5.1 Training Strategy: Episodic Training

Few-shot learning models are **not trained in a traditional way**. Instead, they use **episodic training**, where:

1. Each episode mimics a **few-shot task** with a small support set and a query set.
2. The model is trained on many such episodes to **simulate real test-time conditions**.
3. This helps the model generalize better to unseen classes.

8.5.2 Real-World Use Cases

- **Google's FaceNet (Face Verification)**
  - Uses **Triplet Loss** to compare faces with very few examples.
- **Meta (Facebook) Few-Shot NLP**
  - Uses **MAML-style models** to generalize across new languages.
- **Few-Shot Object Detection (YOLO-FewShot)**
  - Adaptations of **YOLO** for object detection with very limited samples.

8.6 Key Takeaways

- **Few-shot learning enables classification with limited labeled data.**
- **Metric-learning models (Siamese, Triplet, Prototypical) compare samples using embeddings.**
- **Meta-learning (MAML, Reptile) helps models learn how to adapt to new tasks quickly.**
- **Generative models and data augmentation help improve performance in low-data scenarios.**
- **Episodic training is crucial** for improving few-shot learning performance.